# SCAN Disk Scheduling – C Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int a[20], n, head, i, j, t, total = 0;
    int start;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter requests: ");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter head position: ");
    scanf("%d", &head);
    start = head;

    for(i = 0; i < n-1; i++)
        for(j = i+1; j < n; j++)
            if(a[i] > a[j]) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }

    printf("Head movement: %d", head);

    for(i = 0; i < n; i++)
        if(a[i] >= start) {
            total += abs(head - a[i]);
            head = a[i];
            printf(" -> %d", head);
        }

    total += 199 - head;
    head = 199;
    printf(" -> 199");

    for(i = n-1; i >= 0; i--)
        if(a[i] < start) {
            total += abs(head - a[i]);
            head = a[i];
            printf(" -> %d", head);
        }

    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

# SCAN Disk Scheduling – Practical Write-up

## 1. Aim

To write and execute a C program to implement SCAN disk scheduling and calculate total head movement.

## 2. Theory

SCAN is a disk scheduling algorithm, also called the Elevator Algorithm. The disk head moves in one direction, services requests along the way, reaches the end of the disk, and then reverses direction to service the remaining requests.

## 3. Algorithm

1. Start.

2. Read the number of requests, disk requests, and initial head position.

3. Sort the requests in ascending order.

4. Move the head towards the right and service requests in ascending order.

5. Move to the right end of the disk.

6. Reverse direction and service the remaining requests.

7. Calculate and display total head movement.

8. Stop.

### 4. Formula

Total Head Movement = Sum of the absolute differences between consecutive head positions.

THM = Σ |H■■■ – H■|

### 5. Advantages

• Reduces unnecessary disk head movement.

• Provides more uniform waiting times than FCFS in many workloads.

• Services requests in an organized manner.

### 6. Disadvantages

• Requests near the ends may wait longer.

• The head may travel to the disk end even if no request is present there.

### 7. Result

The SCAN disk scheduling algorithm was implemented and executed successfully, and the total head movement was calculated.

### 8. Conclusion

Thus, the SCAN disk scheduling algorithm was studied and implemented in C. It services disk requests by moving in one direction and then reversing direction, helping to organize disk access efficiently.

## Commands to Execute the Program

```
1. mkdir oslab
2. cd oslab
3. Ctrl + Alt + T
4. nano scan.c
5. Press Ctrl + O
6. Press Enter
7. Press Ctrl + X
8. gcc scan.c -o scan
9. ./scan
```